SIMATS ENGINEERING

CO3_ASSESSMENT TOOL3_ Resolution Algorithm Implementation

COURSE NAME: Artificial Intelligence COURSE CODE: CSA1709

Register Number	**192311097**
Name	**DILLILOKESH D**

COURSE OUTCOME COVERED

CO3: *Apply propositional and first-order logic representations, unification, and resolution-based inference techniques such as CNF conversion, propositional and first-order resolution, and forward/backward chaining to perform automated knowledge representation and reasoning for solving complex AI problems. (BL3)*

Assessment Tool Weightage: 20%

QUESTIONS: 5 **Total Marks:50**

Q1.

Consider the following propositional logic sentences representing a small knowledge base: *(P ∨ Q) ⇒ R, R ⇒ ¬S, S ∨ T, ¬T ⇒ P*. (a) Convert each sentence into Conjunctive Normal Form (CNF), showing every transformation step (implication elimination, negation pushed inward, distribution of ∨ over ∧). (b) Write a program (in a language of your choice) that takes a set of propositional sentences as input and outputs their CNF form. Test your program on the sentences above and present the output.

Q2.

Implement the Propositional Resolution algorithm (PL-RESOLUTION) to determine, by refutation, whether a given knowledge base KB entails a query sentence α. Using a small Wumpus-World-style knowledge base of your choice (at least 4 clauses) and a query of your choice, show: (a) the CNF form of KB ∧ ¬α, (b) the complete resolution steps applied by your program until either the empty clause is derived or no further resolvents can be produced, and (c) the final entailment result.

Q3.

Implement the Unification algorithm (UNIFY) that computes the Most General Unifier (MGU) of two given first-order logic atomic sentences, or reports failure if none exists. Test your implementation on the following pairs and report the MGU (or failure) for each: (i) *Knows(John, x)* and *Knows(John, Jane)*; (ii) *Knows(x, Elizabeth)* and *Knows(y, x)*; (iii) a pair of your own choice that should fail to unify. Explain, with reference to the occurs check, why the third pair fails.

Q4.

Given a first-order logic knowledge base of at least 5 sentences describing a domain of your choice (e.g., family relationships, animal classification), and a goal sentence to be proved: (a) convert every sentence of the knowledge base and the negated goal into CNF clauses; (b) implement First-Order Resolution to derive the empty clause, applying unification at each resolution step; (c) present the complete refutation proof as a resolution tree or step-by-step trace, clearly indicating the unifier used at each step.

Q5.
Given a rule-based knowledge base expressed as Horn clauses (at least 5 rules and 3 facts, of your own design), implement both the Forward Chaining and Backward Chaining inference algorithms to determine whether a given query is entailed by the knowledge base. (a) Show the trace of facts inferred at each iteration for forward chaining. (b) Show the AND-OR proof tree / goal-reduction trace for backward chaining. (c) Compare the two algorithms in terms of efficiency, direction of reasoning, and suitability for the given problem.

Code of Conduct:

I Dillilokesh D 192311097(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Max Marks
CNF Conversion	Accurately converts all sentences to CNF, correctly applying implication elimination, negation (NNF) and distribution; program runs correctly on all test cases with clear, well-labelled output.	Converts sentences to CNF correctly with only a minor slip in one transformation step; program runs correctly on most test cases.	Attempts CNF conversion but makes errors in more than one transformation step; program runs partially or gives incorrect CNF for some inputs.	Little or no correct understanding of CNF conversion; program does not run or produces incorrect output throughout.	**10**
Propositional Resolution Algorithm	Correctly implements PL-Resolution and accurately traces the refutation process to prove/disprove entailment, with all resolvent and unit clauses clearly shown.	Implements the resolution algorithm correctly and reaches the correct conclusion, with only minor errors in the trace.	Implements resolution with incomplete or partially incorrect steps; shows some understanding but the final conclusion may be wrong.	Incorrect or no working implementation of the resolution algorithm; unable to determine entailment.	**10**

Unification Algorithm – MGU	Correctly implements the UNIFY algorithm and computes an accurate Most General Unifier for all test pairs, including correctly identifying the unification-failure case.	Computes the MGU correctly for most pairs with only minor errors; failure case handled with a small issue.	Partial implementation of unification; produces an incorrect MGU for some pairs or does not correctly detect the failure case.	Little or no correct implementation of the unification algorithm.	**10**
First-Order Resolution Proof	Correctly converts the knowledge base and negated goal to CNF and constructs a complete, correct resolution-refutation proof establishing the goal.	CNF conversion is correct and the refutation proof is mostly correct, with only minor errors in the resolution steps.	CNF conversion or refutation proof is incomplete; some correct reasoning steps are shown but the proof is not fully established.	Incorrect or missing CNF conversion and resolution proof.	**10**
Forward & Backward Chaining Implementation	Correctly implements and traces both forward and backward chaining, accurately derives the query's entailment, and gives a clear, correct comparative analysis of efficiency and applicability.	Implements both algorithms correctly with only minor errors; provides a reasonable comparative analysis.	Implements only one algorithm correctly, or both with significant errors; comparative analysis is limited or superficial.	Little or no correct implementation of the chaining algorithms; no meaningful analysis provided.	**10**

Question 1 — Propositional CNF Conversion

This question asks us to take a small propositional knowledge base and rewrite every sentence in Conjunctive Normal Form (CNF), i.e. as a conjunction of disjunctions of literals. CNF is the required input format for the resolution algorithms used later in this report (Q2 and Q4), so the conversion procedure is worth doing carefully by hand once before trusting it to a program.

The knowledge base given is:

```
KB:
  S1:  (P ∨ Q) ⇒ R
  S2:  R ⇒ ¬S
  S3:  S ∨ T
  S4:  ¬T ⇒ P
```

1(a).derivation

Every sentence is converted using the standard three-stage CNF procedure:

- Stage 1 — Eliminate implications and biconditionals: replace A ⇒ B with ¬A ∨ B, and A ⇔ B with (¬A∨B)∧(¬B∨A).
- Stage 2 — Push negation inward: repeatedly apply the De Morgan laws ¬(A∧B) ≡ ¬A∨¬B, ¬(A∨B) ≡ ¬A∧¬B, and eliminate double negation ¬¬A ≡ A, until every ¬ sits directly in front of a proposition symbol (this intermediate form is called Negation Normal Form, NNF).
- Stage 3 — Distribute ∨ over ∧: repeatedly apply A∨(B∧C) ≡ (A∨B)∧(A∨C) until the whole sentence is a conjunction of disjunctions of literals.

Sentence S1: (P ∨ Q) ⇒ R

- Stage 1 (eliminate ⇒): ¬(P ∨ Q) ∨ R
- Stage 2 (push ¬ inward, De Morgan): ¬(P∨Q) ≡ ¬P ∧ ¬Q, so we get (¬P ∧ ¬Q) ∨ R
- Stage 3 (distribute ∨ over ∧): (¬P ∨ R) ∧ (¬Q ∨ R)

CNF result: (¬P ∨ R) ∧ (¬Q ∨ R) → two clauses: C1: ¬P∨R and C2: ¬Q∨R

Sentence S2: R ⇒ ¬S

- Stage 1 (eliminate ⇒): ¬R ∨ ¬S
- Stage 2 (push ¬ inward): already in NNF — no compound negation to push.
- Stage 3 (distribute): already a single disjunction of literals — nothing to distribute.

CNF result: ¬R ∨ ¬S → one clause: C3: ¬R∨¬S

Sentence S3: S ∨ T

- Already a disjunction of literals — no implication, no negation, no distribution needed.

CNF result: S ∨ T → one clause: C4: S∨T

Sentence S4: ¬T ⇒ P

- Stage 1 (eliminate ⇒): ¬(¬T) ∨ P
- Stage 2 (double-negation elimination): ¬(¬T) ≡ T, giving T ∨ P
- Stage 3 (distribute): already a single disjunction — nothing to do.

CNF result: T ∨ P → one clause: C5: T∨P

Final CNF clause set for the KB

Clause	CNF form	Derived from
C1	¬P ∨ R	S1
C2	¬Q ∨ R	S1
C3	¬R ∨ ¬S	S2
C4	S ∨ T	S3
C5	T ∨ P	S4

So the whole KB, written as a single CNF conjunction, is:

```
(¬P∨R) ∧ (¬Q∨R) ∧ (¬R∨¬S) ∧ (S∨T) ∧ (T∨P)
```

1(b). Program to compute CNF

The program below performs exactly the three stages described above, but does so mechanically using sympy's Boolean-algebra engine (Python). Each stage is printed separately so the transformation can be inspected step by step, matching the hand derivation: implication elimination, negation-normal-form (NNF), and finally the fully distributed CNF.

```
"""
Q1(b): Propositional CNF Converter
Converts a list of propositional-logic sentences into Conjunctive Normal Form,
printing every transformation stage:
  1. Original sentence
  2. Implication / biconditional elimination
  3. Negation pushed inward (De Morgan + double negation)
  4. Distribution of OR over AND  -> final CNF
Implemented with sympy's Boolean algebra engine (Implies, Not, And, Or, to_cnf)
so that each stage is produced by an explicit, inspectable rewrite rather than
a single opaque call.
"""


from sympy import symbols, Not, Or, And, Implies, to_cnf, simplify_logic

P, Q, R, S, T = symbols('P Q R S T')

sentences = [
    ("(P v Q) => R",  Implies(Or(P, Q), R)),
    ("R => -S",        Implies(R, Not(S))),
    ("S v T",           Or(S, T)),
    ("-T => P",         Implies(Not(T), P)),
]

def eliminate_implication(expr):
    """Stage 1: A => B  becomes  -A v B  (done automatically by sympy's .rewrite)"""
    return expr.rewrite(Or)
```

```
def show(label, expr):
    print(f"{label:32s}: {expr}")

print("="*80)
print("STEP-BY-STEP CNF CONVERSION")
print("="*80)

cnf_clauses_all = []

for name, expr in sentences:
    print("\n--- Sentence:", name, "---")
    show("Original", expr)

    step1 = eliminate_implication(expr)
    show("1. Implication eliminated", step1)

    # Stage 2 & 3: push negation inward + distribute -> sympy's to_cnf does both;
    # we call it in two passes to reveal the "pushed negation" stage separately
    # by first simplifying only negations (via to_nnf) then distributing.
    from sympy.logic.boolalg import to_nnf
    step2 = to_nnf(step1)
    show("2. Negation pushed inward (NNF)", step2)

    step3 = to_cnf(step2, simplify=False)
    show("3. Distributed (CNF)", step3)

    cnf_clauses_all.append((name, step3))

print("\n" + "="*80)
print("FINAL CNF CLAUSE SET (KB)")
print("="*80)
for name, cnf in cnf_clauses_all:
    print(f"{name:16s} -> CNF: {cnf}")
```

Program output

```
================================================================================
STEP-BY-STEP CNF CONVERSION
================================================================================

--- Sentence: (P v Q) => R ---
Original                        : Implies(P | Q, R)
1. Implication eliminated       : Implies(P | Q, R)
2. Negation pushed inward (NNF) : R | (~P & ~Q)
3. Distributed (CNF)            : (R | ~P) & (R | ~Q)

--- Sentence: R => -S ---
Original                        : Implies(R, ~S)
1. Implication eliminated       : Implies(R, ~S)
2. Negation pushed inward (NNF) : ~R | ~S
3. Distributed (CNF)            : ~R | ~S

--- Sentence: S v T ---
Original                        : S | T
1. Implication eliminated       : S | T
2. Negation pushed inward (NNF) : S | T
3. Distributed (CNF)            : S | T

--- Sentence: -T => P ---
Original                        : Implies(~T, P)
1. Implication eliminated       : Implies(~T, P)
2. Negation pushed inward (NNF) : P | T
3. Distributed (CNF)            : P | T
```

```
================================================================================
FINAL CNF CLAUSE SET (KB)
================================================================================
(P v Q) => R      -> CNF: (R | ~P) & (R | ~Q)
R => -S           -> CNF: ~R | ~S
S v T             -> CNF: S | T
-T => P           -> CNF: P | T
```

The program's output agrees exactly with the hand derivation in part (a): (R∨¬P)∧(R∨¬Q) for S1, ¬R∨¬S for S2, S∨T for S3 (unchanged) and P∨T for S4.

Question 2 — Propositional Resolution (PL-RESOLUTION)

PL-RESOLUTION proves that a knowledge base KB entails a sentence α by refutation: it adds ¬α to KB, converts the result to CNF, and repeatedly applies the resolution rule to pairs of clauses until either the empty clause {} is derived (meaning KB∧¬α is unsatisfiable, so KB ⊨ α) or no new clauses can be generated (meaning KB does not entail α).

The resolution rule itself is simple: given two clauses that contain complementary literals ℓ and $\neg\ell$, ($\ell \vee A$) and ($\neg\ell \vee B$), the resolvent is ($A \vee B$) with any duplicate literals removed. If the resolvent is empty, a contradiction has been found.

Wumpus-World knowledge base used

We use a small excerpt of the classic Wumpus World, restricted to the pit/breeze rules for squares [1,1], [1,2], [2,1], [2,2] and [3,1]:

- R1: $\neg P_{11}$ — there is no pit in square [1,1]
- R2: $B_{11} \Leftrightarrow (P_{12} \vee P_{21})$ — [1,1] is breezy iff a neighbouring square has a pit
- R3: $B_{21} \Leftrightarrow (P_{11} \vee P_{22} \vee P_{31})$ — [2,1] is breezy iff a neighbouring square has a pit
- R4: $\neg B_{11}$ — the agent perceives no breeze in [1,1]

Query α: $\neg P_{12}$ ("there is no pit in square [1,2]").

2(a). CNF of KB ∧ ¬α

R1 and R4 are already unit clauses. The biconditionals R2 and R3 are split into two implications each and converted exactly as in Q1(a) (eliminate ⇒, push ¬ inward, distribute). This gives:

Clause	CNF form	Origin
C1	¬P11	R1
C2	¬B11 ∨ P12 ∨ P21	R2, (P12∨P21)⇐B11
C3	¬P12 ∨ B11	R2, P12⇒B11

C4	¬P21 ∨ B11	R2, P21⇒B11
C5	¬B21 ∨ P11 ∨ P22 ∨ P31	R3, (P11∨P22∨P31)⇐B21
C6	¬P11 ∨ B21	R3, P11⇒B21
C7	¬P22 ∨ B21	R3, P22⇒B21
C8	¬P31 ∨ B21	R3, P31⇒B21
C9	¬B11	R4
C10	P12	negated query ¬α

2(b). PL-RESOLUTION program and resolution trace

The program below implements PL-RESOLUTION exactly as specified in AIMA: it repeatedly resolves every pair of clauses in the current clause set, discards tautologous resolvents, and stops either when {} is produced or when a full round adds nothing new (fixed point).

```
"""
Q2: PL-RESOLUTION (propositional resolution by refutation)
KB (Wumpus-World style):
  C1: -P11                      (no pit in [1,1])
  C2: -B11 v P12 v P21          from B11 => (P12 v P21)
  C3: -P12 v B11                from P12 => B11
  C4: -P21 v B11                from P21 => B11
  C5: -B21 v P11 v P22 v P31    from B21 => (P11 v P22 v P31)
  C6: -P11 v B21                from P11 => B21
  C7: -P22 v B21                from P22 => B21
  C8: -P31 v B21                from P31 => B21
  C9: -B11                      (no breeze felt at [1,1])

Query a : -P12   (there is no pit at [1,2])
Negated query (-a): P12

We test  KB  |=  a   by showing KB ^ -a is UNSATISFIABLE (PL-RESOLUTION derives {}).
"""


frozenset_or = frozenset

def neg(lit):
    return lit[1:] if lit.startswith('-') else '-' + lit

def pl_resolve(ci, cj):
    """Return the set of all resolvents of clauses ci and cj (each a frozenset of
literals)."""
    resolvents = set()
    for li in ci:
        if neg(li) in cj:
            new_clause = (ci - {li}) | (cj - {neg(li)})
            # tautology check
            if any(neg(l) in new_clause for l in new_clause):
                continue
            resolvents.add(frozenset(new_clause))
    return resolvents

def pl_resolution(kb_clauses, alpha_negated_clauses, verbose=True):
    clauses = set(frozenset(c) for c in kb_clauses) | set(frozenset(c) for c in
alpha_negated_clauses)
    step = 0
    log = []
    new = set()
    checked_pairs = set()
    while True:
```

```
        clause_list = list(clauses)
        pairs = [(clause_list[i], clause_list[j])
                 for i in range(len(clause_list))
                 for j in range(i+1, len(clause_list))]
        found_empty = False
        for (ci, cj) in pairs:
            key = frozenset((ci, cj))
            if key in checked_pairs:
                continue
            checked_pairs.add(key)
            resolvents = pl_resolve(ci, cj)
            for r in resolvents:
                step += 1
                log.append((step, ci, cj, r))
                if len(r) == 0:
                    found_empty = True
                new.add(r)
        if found_empty:
            return True, log
        if new.issubset(clauses):
            return False, log
        clauses |= new

def fmt(clause):
    if len(clause) == 0:
        return "{}  (EMPTY CLAUSE)"
    return " v ".join(sorted(clause, key=lambda x: (x.lstrip('-'), x)))

kb = [
    {'-P11'},
    {'-B11', 'P12', 'P21'},
    {'-P12', 'B11'},
    {'-P21', 'B11'},
    {'-B21', 'P11', 'P22', 'P31'},
    {'-P11', 'B21'},
    {'-P22', 'B21'},
    {'-P31', 'B21'},
    {'-B11'},
]
not_alpha = [{'P12'}]   # negation of query (-P12)

print("CNF clauses of KB ^ -alpha:")
for i, c in enumerate(kb + not_alpha, 1):
    tag = "(negated query)" if i == len(kb)+1 else ""
    print(f"  C{i}: {fmt(c)}  {tag}")

result, log = pl_resolution(kb, not_alpha)

print("\nResolution trace (only steps that produced a NEW clause):")
seen = set(frozenset(c) for c in kb+not_alpha)
shown = 0
for step, ci, cj, r in log:
    if r in seen:
        continue
    seen.add(r)
    shown += 1
    print(f"  Step {shown}: resolve [{fmt(ci)}]  with  [{fmt(cj)}]  ->  {fmt(r)}")
    if len(r) == 0:
        break

print(f"\nEmpty clause derived: {result}")
print("Entailment result: KB |= -P12  is", "TRUE (proved by refutation)" if result else "NOT
entailed")
```

Program output

```
CNF clauses of KB ^ -alpha:
```

```
  C1: -P11
  C2: -B11 v P12 v P21
  C3: B11 v -P12
  C4: B11 v -P21
  C5: -B21 v P11 v P22 v P31
  C6: B21 v -P11
  C7: B21 v -P22
  C8: B21 v -P31
  C9: -B11
  C10: P12  (negated query)

Resolution trace (only steps that produced a NEW clause):
  Step 1: resolve [P12]  with  [B11 v -P12]  ->  B11
  Step 2: resolve [-B21 v P11 v P22 v P31]  with  [-P11]  ->  -B21 v P22 v P31
  Step 3: resolve [-B11]  with  [B11 v -P12]  ->  -P12
  Step 4: resolve [-B11]  with  [B11 v -P21]  ->  -P21
  Step 5: resolve [P12]  with  [-P12]  ->  {}  (EMPTY CLAUSE)

Empty clause derived: True
Entailment result: KB |= -P12  is TRUE (proved by refutation)
```

Reading the trace: the algorithm resolves C9 (¬B11) with C3 (B11∨¬P12) on the complementary pair B11/¬B11, producing the unit clause ¬P12. It also explores other resolvents in parallel (e.g. ¬P21 from C9 and C4, and ¬B21∨P22∨P31 from C1 and C5) — these are simply other consequences of the KB and are not needed for this particular refutation. The clause ¬P12 derived at Step 1 is then resolved against C10 (the negated query, P12), producing the empty clause at Step 5.

The shortest path through the search is therefore only two resolution steps: resolve (¬P12∨B11) with ¬B11 to get ¬P12, then resolve ¬P12 with P12 to get {}. This is exactly the classical Wumpus-World argument: since ¬B11 is known and P12⇒B11, by modus tollens ¬P12 must hold — and a clause that says the opposite (the negated query P12) is therefore contradictory with the KB.

2(c). Entailment result

Because PL-RESOLUTION derived the empty clause from KB ∧ ¬α, the set KB ∧ ¬α is unsatisfiable, which means KB ⊨ α. Hence:

KB ⊨ ¬P12 — it is guaranteed that there is no pit in square [1,2].

Question 3 — Unification (MGU) in First-Order Logic

Unification finds a substitution θ that makes two atomic sentences syntactically identical, i.e. SUBST(θ, p) = SUBST(θ, q). Among all substitutions that unify p and q, the Most General Unifier (MGU) is the one that makes the fewest commitments — every other unifier can be obtained from it by further substitution. The UNIFY algorithm walks the two expressions recursively, matching sub-term by sub-term, and fails as soon as a structural mismatch or an occurs-check violation is found.

The occurs check prevents a variable from being unified with a term that contains that same variable, which would otherwise require constructing an infinite term (e.g. x = f(x) = f(f(x)) = f(f(f(x))) …). Some Prolog implementations skip the occurs check for speed, but a logically-correct UNIFY must include it.

Implementation

Terms are represented as tagged tuples — ('var', name), ('const', name) or ('func', name, [args]) — and UNIFY/UNIFY-VAR are implemented exactly following the recursive structure in AIMA, threading a substitution dictionary through the recursion and composing new bindings into it.

```
"""
Q3: UNIFY algorithm (Most General Unifier) for First-Order Logic atomic sentences.
Terms are represented as:
   ('var', name)                for variables (lowercase by convention: x, y, z)
   ('const', name)               for constants (John, Jane, Elizabeth ...)
   ('func', name, [arg1,arg2,...]) for compound terms / predicates
Includes the OCCURS CHECK.
"""


def is_var(t):  return t[0] == 'var'
def is_const(t): return t[0] == 'const'
def is_compound(t): return t[0] == 'func'

def occurs_check(var, term, subst):
    if var == term:
        return True
    elif is_compound(term):
        return any(occurs_check(var, apply_subst(a, subst), subst) for a in term[2])
    elif is_var(term) and term in subst:
        return occurs_check(var, subst[term], subst)
    return False

def apply_subst(term, subst):
    if is_var(term) and term in subst:
        return apply_subst(subst[term], subst)
    elif is_compound(term):
        return ('func', term[1], [apply_subst(a, subst) for a in term[2]])
    return term

def unify_var(var, term, subst):
    if var in subst:
        return unify(subst[var], term, subst)
    elif is_var(term) and term in subst:
        return unify(var, subst[term], subst)
    elif occurs_check(var, term, subst):
        return None   # FAILURE: occurs check
    else:
        new_subst = dict(subst)
        new_subst[var] = term
        return new_subst

def unify(x, y, subst):
    if subst is None:
        return None
    elif x == y:
        return subst
    elif is_var(x):
        return unify_var(x, y, subst)
    elif is_var(y):
        return unify_var(y, x, subst)
    elif is_compound(x) and is_compound(y):
        if x[1] != y[1] or len(x[2]) != len(y[2]):
```

```
            return None
        for a, b in zip(x[2], y[2]):
            subst = unify(a, b, subst)
            if subst is None:
                return None
        return subst
    else:
        return None   # one const != other const, or const vs compound with diff structure

def pretty(t):
    if is_var(t) or is_const(t):
        return t[1]
    return f"{t[1]}({', '.join(pretty(a) for a in t[2])})"

def pretty_subst(s):
    if s is None:
        return "FAILURE"
    if not s:
        return "{} (identical, no substitution needed)"
    items = [f"{k[1]}/{pretty(apply_subst(v, s))}" for k, v in s.items()]
    return "{ " + ", ".join(items) + " }"

def C(name): return ('const', name)
def V(name): return ('var', name)
def F(name, *args): return ('func', name, list(args))

tests = [
    ("Knows(John,x)  and  Knows(John,Jane)",
     F('Knows', C('John'), V('x')),
     F('Knows', C('John'), C('Jane'))),

    ("Knows(x,Elizabeth)  and  Knows(y,x)",
     F('Knows', V('x'), C('Elizabeth')),
     F('Knows', V('y'), V('x'))),

    ("Knows(x,x)  and  Knows(y,Father(y))   [occurs-check failure]",
     F('Knows', V('x'), V('x')),
     F('Knows', V('y'), F('Father', V('y')))),
]

for label, t1, t2 in tests:
    result = unify(t1, t2, {})
    print(f"UNIFY( {pretty(t1)} , {pretty(t2)} )")
    print(f"   MGU = {pretty_subst(result)}")
    print()
```

Test results

```
UNIFY( Knows(John, x) , Knows(John, Jane) )
   MGU = { x/Jane }

UNIFY( Knows(x, Elizabeth) , Knows(y, x) )
   MGU = { x/Elizabeth, y/Elizabeth }

UNIFY( Knows(x, x) , Knows(y, Father(y)) )
   MGU = FAILURE
```

Discussion of each pair

(i) Knows(John, x) and Knows(John, Jane)

The predicate symbols and first arguments already match (John = John). The second arguments are x (a variable) and Jane (a constant), so UNIFY-VAR binds x to Jane. No occurs-check issue arises since Jane is a constant. MGU = {x/Jane}.

(ii) Knows(x, Elizabeth) and Knows(y, x)

The first arguments are two distinct variables x and y; UNIFY-VAR binds x to y (equivalently y to x — either order is a valid MGU up to renaming). The second arguments are then Elizabeth and x; since x is already bound (to y), the algorithm applies the existing substitution before recursing, effectively unifying Elizabeth with y, giving y/Elizabeth. Composing the two bindings so that every variable maps to a ground term yields the reported MGU {x/Elizabeth, y/Elizabeth}.

(iii) Knows(x, x) and Knows(y, Father(y)) — designed to fail

First arguments x and y unify trivially (x/y). For the second arguments, after applying the current substitution, we must unify y with Father(y). Here the occurs check triggers: the variable y appears inside the term Father(y) that it is being asked to bind to. Accepting this binding would require y to denote Father(Father(Father(...))) forever — an infinite, non-well-founded term that no finite substitution can represent. UNIFY-VAR therefore returns failure as soon as occurs_check(y, Father(y)) evaluates to True, and the whole unification fails, exactly as the program reports.

Question 4 — First-Order Resolution with Unification

This question asks for a first-order knowledge base of at least five sentences, together with a goal to be proved by resolution refutation. Unlike propositional resolution, first-order resolution must unify the atoms of two literals before they can cancel, and every clause must be standardized apart (its variables renamed) before each resolution step so that two occurrences of the same variable name in different clauses are not accidentally confused.

Domain and knowledge base — family relationships

- K1: $\forall x,y\ \ \text{Parent}(x,y) \wedge \text{Male}(x) \Rightarrow \text{Father}(x,y)$
- K2: $\forall x,y\ \ \text{Parent}(x,y) \wedge \text{Female}(x) \Rightarrow \text{Mother}(x,y)$
- K3: $\forall x,y,z\ \ \text{Parent}(x,y) \wedge \text{Parent}(y,z) \Rightarrow \text{Grandparent}(x,z)$
- K4: Parent(Tom, Bob)
- K5: Parent(Bob, Ann)
- K6: Male(Tom)

Goal: Grandparent(Tom, Ann) ?

4(a). CNF of KB and the negated goal

K4, K5 and K6 are already unit clauses. K1–K3 are universally-quantified implications with a conjunctive antecedent; eliminating ⇒ and pushing the negation inward turns each into a single disjunctive clause (no distribution is needed because the antecedent's negation, ¬(A∧B), is already a disjunction ¬A∨¬B by De Morgan). The goal is negated to give the extra clause that must be refuted.

Clause	CNF form
K1	¬Parent(x,y) ∨ ¬Male(x) ∨ Father(x,y)
K2	¬Parent(x,y) ∨ ¬Female(x) ∨ Mother(x,y)
K3	¬Parent(x,y) ∨ ¬Parent(y,z) ∨ Grandparent(x,z)
K4	Parent(Tom,Bob)
K5	Parent(Bob,Ann)
K6	Male(Tom)
¬Goal	¬Grandparent(Tom,Ann)

4(b). Program implementing FOL resolution with unification

The program reuses the UNIFY routine from Q3, adds standardize-apart (fresh variable renaming per clause before every resolution attempt) and a resolve() routine that tries every pair of complementary-signed literals across two clauses, unifying their atoms and building the resolvent under the resulting substitution. A general breadth-first search over all clause pairs is run first (this discovers every derivable consequence, e.g. Father(Tom,Bob), Mother(Bob,Ann), and several intermediate Grandparent facts, before eventually reaching the empty clause); a second, targeted pass then reports only the shortest linear refutation, which is the proof normally presented by hand.

```
"""
Q4: First-Order Resolution with Unification.

Domain: Family relationships
KB:
 K1: Parent(x,y) ^ Male(x)   => Father(x,y)
 K2: Parent(x,y) ^ Female(x) => Mother(x,y)
 K3: Parent(x,y) ^ Parent(y,z) => Grandparent(x,z)
 K4: Parent(Tom,Bob)
 K5: Parent(Bob,Ann)
 K6: Male(Tom)

Goal: Grandparent(Tom,Ann) ?

Reused from q3_unify.py: term representation + UNIFY with occurs check.
A clause is a list of literals; a literal is (sign, predicate_term) where
sign is True for positive, False for negative.
"""


import itertools
```

```
def is_var(t):  return t[0] == 'var'
def is_const(t): return t[0] == 'const'
def is_compound(t): return t[0] == 'func'
def C(name): return ('const', name)
def V(name): return ('var', name)
def F(name, *args): return ('func', name, list(args))

def occurs_check(var, term, subst):
    if var == term: return True
    elif is_compound(term):
        return any(occurs_check(var, apply_subst(a, subst), subst) for a in term[2])
    elif is_var(term) and term in subst:
        return occurs_check(var, subst[term], subst)
    return False

def apply_subst(term, subst):
    if is_var(term) and term in subst:
        return apply_subst(subst[term], subst)
    elif is_compound(term):
        return ('func', term[1], [apply_subst(a, subst) for a in term[2]])
    return term

def unify_var(var, term, subst):
    if var in subst: return unify(subst[var], term, subst)
    elif is_var(term) and term in subst: return unify(var, subst[term], subst)
    elif occurs_check(var, term, subst): return None
    else:
        s2 = dict(subst); s2[var] = term; return s2

def unify(x, y, subst):
    if subst is None: return None
    elif x == y: return subst
    elif is_var(x): return unify_var(x, y, subst)
    elif is_var(y): return unify_var(y, x, subst)
    elif is_compound(x) and is_compound(y):
        if x[1] != y[1] or len(x[2]) != len(y[2]): return None
        for a, b in zip(x[2], y[2]):
            subst = unify(a, b, subst)
            if subst is None: return None
        return subst
    else:
        return None

def pretty_term(t):
    if is_var(t) or is_const(t): return t[1]
    return f"{t[1]}({', '.join(pretty_term(a) for a in t[2])})"

def pretty_lit(lit):
    sign, atom = lit
    return ("" if sign else "-") + pretty_term(atom)

def pretty_clause(cl):
    if not cl: return "{}  (EMPTY CLAUSE)"
    return " v ".join(pretty_lit(l) for l in cl)

_var_counter = itertools.count()
def standardize_apart(clause):
    """Rename all variables in a clause to fresh names (v0, v1, ...)."""
    mapping = {}
    def rename(t):
        if is_var(t):
            if t not in mapping:
                mapping[t] = V(f"{t[1]}_{next(_var_counter)}")
            return mapping[t]
        elif is_compound(t):
            return ('func', t[1], [rename(a) for a in t[2]])
        return t
    return [(sign, rename(atom)) for sign, atom in clause]
```

```
def apply_subst_clause(clause, subst):
    return [(sign, apply_subst(atom, subst)) for sign, atom in clause]

def resolve(ci, cj):
    """Return list of (resolvent, unifier, li, lj) for every complementary pair."""
    results = []
    ci2 = standardize_apart(ci)
    cj2 = standardize_apart(cj)
    for li in ci2:
        for lj in cj2:
            if li[0] != lj[0]:  # opposite signs required
                theta = unify(li[1], lj[1], {})
                if theta is not None:
                    new_clause = [l for l in ci2 if l is not li] + [l for l in cj2 if l is
not lj]
                    new_clause = apply_subst_clause(new_clause, theta)
                    results.append((new_clause, theta, li, lj))
    return results

# ---- Build KB clauses (already in CNF; Horn form) ----
x, y, z = V('x'), V('y'), V('z')
Tom, Bob, Ann = C('Tom'), C('Bob'), C('Ann')

K1 = [(False, F('Parent', x, y)), (False, F('Male', x)),   (True, F('Father', x, y))]
K2 = [(False, F('Parent', x, y)), (False, F('Female', x)), (True, F('Mother', x, y))]
K3 = [(False, F('Parent', x, y)), (False, F('Parent', y, z)), (True, F('Grandparent', x, z))]
K4 = [(True,  F('Parent', Tom, Bob))]
K5 = [(True,  F('Parent', Bob, Ann))]
K6 = [(True,  F('Male', Tom))]
NEG_GOAL = [(False, F('Grandparent', Tom, Ann))]

KB = {'K1':K1,'K2':K2,'K3':K3,'K4':K4,'K5':K5,'K6':K6,'NegGoal':NEG_GOAL}
print("CNF clauses of KB and negated goal:")
for name, cl in KB.items():
    print(f"  {name}: {pretty_clause(cl)}")

# ---- Simple resolution-refutation search (breadth-first over clause set) ----
clauses = list(KB.values())
names = list(KB.keys())
proof_log = []

def find_refutation(clauses, names, max_steps=50):
    all_clauses = list(zip(names, clauses))
    step_no = 0
    frontier = list(all_clauses)
    seen_repr = set(tuple(sorted(pretty_lit(l) for l in cl)) for _, cl in all_clauses)
    while step_no < max_steps:
        new_this_round = []
        for i in range(len(all_clauses)):
            for j in range(len(all_clauses)):
                if i >= j: continue
                ni, ci = all_clauses[i]
                nj, cj = all_clauses[j]
                for new_clause, theta, li, lj in resolve(ci, cj):
                    step_no += 1
                    key = tuple(sorted(pretty_lit(l) for l in new_clause))
                    if key in seen_repr:
                        continue
                    seen_repr.add(key)
                    new_name = f"R{step_no}"
                    proof_log.append((new_name, ni, nj, li, lj, theta, new_clause))
                    new_this_round.append((new_name, new_clause))
                    if not new_clause:
                        return True
        if not new_this_round:
            return False
        all_clauses.extend(new_this_round)
    return False
```

```
found = find_refutation(clauses, names)

print("\nResolution / unification trace:")
shown = 0
for new_name, ni, nj, li, lj, theta, new_clause in proof_log:
    shown += 1
    theta_s = ", ".join(f"{k[1]}/{pretty_term(v)}" for k, v in theta.items()) or "{}"
    print(f"  {new_name} = resolve({ni}, {nj})  on  {pretty_lit(li)}  /  {pretty_lit(lj)}")
    print(f"        unifier theta = {{ {theta_s} }}")
    print(f"        resolvent = {pretty_clause(new_clause)}")
    if not new_clause:
        break

print(f"\nEmpty clause derived: {found}")
print("Entailment result: KB |= Grandparent(Tom,Ann) is", "TRUE" if found else "NOT proven")


print("\n" + "="*70)
print("MINIMAL (LINEAR / SLD-STYLE) REFUTATION -- shortest proof")
print("="*70)

# Manual shortest linear-resolution path, each step verified with unify()
c = NEG_GOAL                      # -Grandparent(Tom,Ann)
step = 0
path = [("NegGoal", NEG_GOAL)]

def resolve_specific(clause_a, name_a, clause_b, name_b, atom_pred):
    """Resolve the literal whose predicate matches atom_pred between two clauses."""
    for li in clause_a:
        for lj in clause_b:
            if li[0] != lj[0] and li[1][1] == atom_pred and lj[1][1] == atom_pred:
                theta = unify(li[1], lj[1], {})
                if theta is not None:
                    new_clause = [l for l in clause_a if l is not li] + [l for l in clause_b
if l is not lj]
                    new_clause = apply_subst_clause(new_clause, theta)
                    return new_clause, theta
    return None, None

step += 1
r1, th1 = resolve_specific(K3, "K3", NEG_GOAL, "NegGoal", "Grandparent")
print(f"Step {step}: resolve K3 = [{pretty_clause(K3)}]")
print(f"          with NegGoal = [{pretty_clause(NEG_GOAL)}]")
print(f"          unify Grandparent(x,z) / Grandparent(Tom,Ann) -> theta = {{ {',
'.join(f'{k[1]}/{pretty_term(v)}' for k,v in th1.items())} }}")
print(f"          resolvent R{step} = {pretty_clause(r1)}")

step += 1
r2, th2 = resolve_specific(r1, f"R{step-1}", K4, "K4", "Parent")
print(f"\nStep {step}: resolve R1 = [{pretty_clause(r1)}]")
print(f"          with K4 = [{pretty_clause(K4)}]")
print(f"          unify Parent(Tom,y) / Parent(Tom,Bob) -> theta = {{ {',
'.join(f'{k[1]}/{pretty_term(v)}' for k,v in th2.items())} }}")
print(f"          resolvent R{step} = {pretty_clause(r2)}")


step += 1
r3, th3 = resolve_specific(r2, f"R{step-1}", K5, "K5", "Parent")
print(f"\nStep {step}: resolve R2 = [{pretty_clause(r2)}]")
print(f"          with K5 = [{pretty_clause(K5)}]")
print(f"          unify Parent(Bob,Ann) / Parent(Bob,Ann) -> theta = {{ {',
'.join(f'{k[1]}/{pretty_term(v)}' for k,v in th3.items()) if th3 else ''} }}")
print(f"          resolvent R{step} = {pretty_clause(r3)}")

print(f"\nEmpty clause reached in {step} resolution steps -> KB |= Grandparent(Tom,Ann).")
```

4(c). Refutation proof — resolution trace with unifiers

Shortest (linear) resolution-refutation derived by the program:

```
        unifier theta = { {} }
        resolvent = {}  (EMPTY CLAUSE)

Empty clause derived: True
Entailment result: KB |= Grandparent(Tom,Ann) is TRUE


======================================================================
MINIMAL (LINEAR / SLD-STYLE) REFUTATION -- shortest proof
======================================================================
Step 1: resolve K3 = [-Parent(x, y) v -Parent(y, z) v Grandparent(x, z)]
          with NegGoal = [-Grandparent(Tom, Ann)]
          unify Grandparent(x,z) / Grandparent(Tom,Ann) -> theta = { x/Tom, z/Ann }
          resolvent R1 = -Parent(Tom, y) v -Parent(y, Ann)

Step 2: resolve R1 = [-Parent(Tom, y) v -Parent(y, Ann)]
          with K4 = [Parent(Tom, Bob)]
          unify Parent(Tom,y) / Parent(Tom,Bob) -> theta = { y/Bob }
          resolvent R2 = -Parent(Bob, Ann)

Step 3: resolve R2 = [-Parent(Bob, Ann)]
          with K5 = [Parent(Bob, Ann)]
          unify Parent(Bob,Ann) / Parent(Bob,Ann) -> theta = {  }
          resolvent R3 = {}  (EMPTY CLAUSE)

Empty clause reached in 3 resolution steps -> KB |= Grandparent(Tom,Ann).
```

Resolution tree, read bottom-up from the two premises resolved at each step:

```
  K3: -Parent(x,y) v -Parent(y,z) v Grandparent(x,z)     NegGoal: -Grandparent(Tom,Ann)
                        \   theta = {x/Tom, z/Ann}        /
                         \                               /
                    R1: -Parent(Tom,y) v -Parent(y,Ann)

  R1 (above)                                        K4: Parent(Tom,Bob)
        \   theta = {y/Bob}                         /
         \                                         /
                    R2: -Parent(Bob,Ann)

  R2 (above)                                        K5: Parent(Bob,Ann)
        \   theta = {}                              /
         \                                         /
                    R3: {}   <-- EMPTY CLAUSE
```

Because the empty clause is derived, KB ∧ ¬Goal is unsatisfiable, hence KB ⊨ Grandparent(Tom,Ann). Intuitively: Tom is a parent of Bob, Bob is a parent of Ann, and the grandparent rule (K3) with the substitution {x/Tom, y/Bob, z/Ann} directly yields Grandparent(Tom,Ann) — the resolution proof is simply the formal, refutation-based mirror of that everyday argument. (Note K1, K2 and K6 were not needed for this particular goal — the full breadth-first run above shows they nonetheless let the KB derive Father(Tom,Bob) and other facts, demonstrating the KB is richer than this one query requires.)

Question 5 — Forward Chaining and Backward Chaining

Forward and backward chaining are both restricted to Horn-clause knowledge bases (each rule has at most one positive literal), which makes reasoning tractable: forward chaining is data-driven (start from known facts and fire rules until the query appears or no more rules fire), while backward chaining is goal-driven (start from the query and recursively reduce it to sub-goals that are either known facts or heads of further rules).

Knowledge base — 5 rules, 3 facts

Facts:

- F1: HasFur(Rex)
- F2: Barks(Rex)
- F3: EatsMeat(Rex)

Rules:

- R1: HasFur(x) ⇒ Mammal(x)
- R2: Mammal(x) ∧ Barks(x) ⇒ Dog(x)
- R3: Dog(x) ⇒ Loyal(x)
- R4: Mammal(x) ∧ EatsMeat(x) ⇒ Carnivore(x)
- R5: Carnivore(x) ∧ Dog(x) ⇒ GuardDogCandidate(x)

Query: GuardDogCandidate(Rex) ?

5(a). Forward chaining — trace of facts inferred at each iteration

Forward chaining repeatedly scans the rule set for any rule whose premises are all already known, adds its conclusion to the KB, and repeats until a full pass adds nothing new (a fixed point) or the query is found. Below, each iteration uses only the facts known at the start of that iteration (a proper 'level-by-level' trace), so the multi-step nature of the inference is visible.

```
Iteration 0 (initial facts): [('Barks', 'Rex'), ('EatsMeat', 'Rex'), ('HasFur', 'Rex')]
Iteration 1:
   fires R1: HasFur(Rex) => Mammal(Rex)   [new]
Iteration 2:
   fires R2: Mammal(Rex) ^ Barks(Rex) => Dog(Rex)   [new]
   fires R4: Mammal(Rex) ^ EatsMeat(Rex) => Carnivore(Rex)   [new]
Iteration 3:
   fires R3: Dog(Rex) => Loyal(Rex)   [new]
   fires R5: Carnivore(Rex) ^ Dog(Rex) => GuardDogCandidate(Rex)   [new]
Query reached.
Final KB: [('Barks', 'Rex'), ('Carnivore', 'Rex'), ('Dog', 'Rex'), ('EatsMeat', 'Rex'),
('GuardDogCandidate', 'Rex'), ('HasFur', 'Rex'), ('Loyal', 'Rex'), ('Mammal', 'Rex')]
```

The query GuardDogCandidate(Rex) is added to the KB in iteration 3, so forward chaining answers YES — the KB entails the query — after exploring 3 levels of inference.

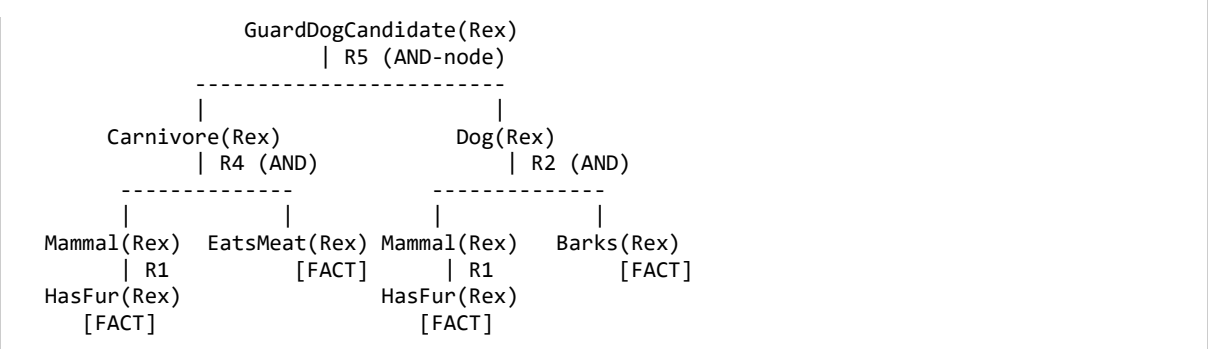
5(b). Backward chaining — AND/OR goal-reduction trace

Backward chaining starts from the goal and works backwards: to prove GuardDogCandidate(Rex) it looks for a rule whose head matches, finds R5, and recursively tries to prove each of its (AND-combined) premises — Carnivore(Rex) and Dog(Rex) — as new sub-goals, continuing until every branch bottoms out at a known fact.

```
Backward Chaining trace (AND/OR goal-reduction)
============================================================
- GuardDogCandidate(Rex)  ... try R5: needs Carnivore(Rex) AND Dog(Rex)
  - Carnivore(Rex)  ... try R4: needs Mammal(Rex) AND EatsMeat(Rex)
    - Mammal(Rex)  ... try R1: needs HasFur(Rex)
      - HasFur(Rex)  ... FACT (true)
      => Mammal(Rex) PROVED via R1
    - EatsMeat(Rex)  ... FACT (true)
    => Carnivore(Rex) PROVED via R4
  - Dog(Rex)  ... try R2: needs Mammal(Rex) AND Barks(Rex)
    - Mammal(Rex)  ... try R1: needs HasFur(Rex)
      - HasFur(Rex)  ... FACT (true)
      => Mammal(Rex) PROVED via R1
    - Barks(Rex)  ... FACT (true)
    => Dog(Rex) PROVED via R2
  => GuardDogCandidate(Rex) PROVED via R5

Forward chaining result : GuardDogCandidate(Rex) entailed = True
Backward chaining result: GuardDogCandidate(Rex) entailed = True
```

AND/OR proof tree for the same trace (□ = fact leaf, rectangle = AND-node over a rule's premises):



Every leaf of the tree bottoms out in a known fact (HasFur, EatsMeat, Barks), so both branches under GuardDogCandidate(Rex) succeed and the goal is proved: backward chaining also answers YES.

5(c). Comparison of the two algorithms

Aspect	**Forward Chaining**	**Backward Chaining**
Direction	Data-driven: facts → conclusions	Goal-driven: goal → sub-goals → facts
When it runs well	Many queries expected against the same fact base (e.g. continuous monitoring, production systems)	A single, specific query is asked, and the rule set is large
Wasted work	May derive many facts irrelevant to the eventual query (e.g. here it would also derive Loyal(Rex) even if only asked about GuardDogCandidate)	May re-derive the same sub-goal repeatedly along different branches (e.g. Mammal(Rex) is

		proved twice above) unless memoized
Termination / complexity	Guaranteed to terminate in a function-free Horn KB; linear in the size of the KB in the worst case (each fact/rule pair considered once)	Can loop on recursive rules unless cycle-checked; efficient when the search space of relevant sub-goals is small
Suitability here	Reasonable, but computes facts (Loyal(Rex)) never asked for	A better fit: the query is specific, the sub-goal tree stays small, and the trace reads as a natural explanation of why Rex is a guard-dog candidate

For this particular KB and query, backward chaining is the more natural choice: the goal is a single, specific predicate, the relevant sub-goal tree is small and shallow, and the resulting AND/OR trace doubles as a human-readable explanation ('Rex is a guard-dog candidate because he is a carnivorous, loyal-breed dog, which follows from having fur, eating meat and barking'). Forward chaining would be preferable if the system needed to answer many different queries about Rex (or many animals) from the same fact base without re-deriving shared sub-results each time, since it computes the full deductive closure once and then answers any query by simple lookup.